

```
!pip install -U transformers
```

Show hidden output

```
!pip install -U datasets
```

Show hidden output

Local Inference on GPU

Model page: <https://huggingface.co/casperhansen/llama-3-8b-instruct-awq>

⚠️ If the generated code snippets do not work, please open an issue on either the [model repo](#) and/or on [huggingface.js](#) 🙏

```
!pip install gptqmodel
```

Show hidden output

```
!uv pip install vllm==0.18.0 --torch-backend=auto
```

Show hidden output

```
import os
import sys
import contextlib
# Required for runtime swap
os.environ["VLLM_ENABLE_V1_MULTIPROCESSING"] = "0"

# Import vLLM's system utils explicitly
import vllm.utils.system_utils

# Create a dummy context manager that skips the fileno() logic
@contextlib.contextmanager
def colab_safe_suppress():
    yield # Do absolutely nothing

# Neutralize vLLM's native suppress_stdout with our dummy function
vllm.utils.system_utils.suppress_stdout = colab_safe_suppress
```

```
# Library imports
from vllm import LLM, SamplingParams
import torch
import torch.nn.functional as F
import triton
import triton.language as tl
import torch.nn as nn
```

```
from vllm.model_executor.models.llama import LlamaMLP
from vllm.model_executor.layers.layernorm import RMSNorm
```

WARNING:torchao:Skipping import of cpp extensions due to incompatible torch version. Please upgrade to torch >= 2.11.0 (found 2.

```
# Set runtime device
DEVICE = triton.runtime.driver.active.get_active_torch_device()
```

```
@triton.jit
def swiglu(x_ptr, out_ptr, token_stride, hidden_dim_stride, out_stride_x, out_stride_y, num_tokens, hidden_dim, block_size_m: t
"""
    Kernel which applies the SwiGLU activation function to the input matrix.

    Input parameters:
    x_ptr: Pointer to the input matrix.
    out_ptr: Pointer to the output matrix.
    token_stride: Stride between consecutive tokens in the input matrix.
    hidden_dim_stride: Stride between consecutive hidden dimensions in the input matrix.
    out_stride_x: Stride between consecutive tokens in the output matrix.
    out_stride_y: Stride between consecutive hidden dimensions in the output matrix.
    num_tokens: Number of tokens in the input matrix.
    hidden_dim: Number of hidden dimensions in the input matrix.
```

```

block_size_m: Block size for the number of tokens.
block_size_k: Block size for the number of hidden dimensions.
"""

# Get 1D block id
pid = tl.program_id(axis = 0)

# Thread id with which block starts
block_start = pid * block_size_m

# Get the offsets for the number of tokens for a specific block
token_offsets = block_start + tl.arange(0, block_size_m)

# Get the offsets for the hidden dimension. Since each block has to apply it on the entire vector, it starts from zero.
hidden_dim_offsets = tl.arange(0, block_size_k)

# Mask for handling the final block
token_mask = token_offsets < num_tokens

# Calculate the offsets for the gate and up matrices in 2D.
gate_offsets = token_offsets[:, None] * token_stride + hidden_dim_offsets[None, :] * hidden_dim_stride
up_offsets = token_offsets[:, None] * token_stride + (hidden_dim + hidden_dim_offsets)[None, :] * hidden_dim_stride

# Calculate offsets for writing to the output matrix.
output_offsets = token_offsets[:, None] * out_stride_x + hidden_dim_offsets[None, :] * out_stride_y

# Loop over the hidden dimension
for col_group in range(tl.cdiv(hidden_dim, block_size_k)):
    # Mask for handling when offsets go beyond the hidden dimension
    hidden_dim_mask = hidden_dim_offsets < hidden_dim

    # Load the actual values
    gate_mat = tl.load(x_ptr + gate_offsets, mask = token_mask[:, None] & hidden_dim_mask[None, :], other = 0.0)
    up_mat = tl.load(x_ptr + up_offsets, mask = token_mask[:, None] & hidden_dim_mask[None, :], other = 0.0)

    # Upcasting from float16 to float32 to prevent underflow
    gate_mat = gate_mat.to(tl.float32)
    up_mat = up_mat.to(tl.float32)

    # SwiGLU activation
    out_mat_fp32 = gate_mat * tl.sigmoid(gate_mat) * up_mat

    # Downcast output to match model architecture
    out_mat = out_mat_fp32.to(tl.float16)

    # Store it in the output matrix
    tl.store(out_ptr + output_offsets, out_mat, mask = token_mask[:, None] & hidden_dim_mask[None, :])

    # Step over each offset
    hidden_dim_offsets += block_size_k
    gate_offsets += block_size_k * hidden_dim_stride
    up_offsets += block_size_k * hidden_dim_stride
    output_offsets += block_size_k * out_stride_y

```

```

class customSwiglu(nn.Module):
    """Class which calls the kernel."""

    def __init__(self):
        """Init function."""
        super().__init__()

    def forward(self, x):
        """Forward method which invokes the kernel."""
        # Save the original shape for the output matrix
        original_shape = x.shape

        # Collapse the first two dimensions to maintain token level dimensionality.
        squashed_x = x.view([-1, x.shape[-1]])

        # Total number of tokens being passed
        total_tokens = squashed_x.shape[0]

        # Since vLLM passes the gate and up matrices in a single matrix by concatenating them side by side, the column width is twice
        hidden_dim = squashed_x.shape[1] // 2

        # Initialize the output matrix

```

```

output = torch.empty((total_tokens, hidden_dim), device = x.device, dtype = x.dtype)

# Design a 1D grid of blocks
grid = lambda meta: ((triton.cdiv(total_tokens, meta['block_size_m'])),)

# Invoke the kernel
swiglu[grid](squashed_x, output, squashed_x.stride(0), squashed_x.stride(1), output.stride(0), output.stride(1), total_tokens)

# Get the shape based on the original hidden dimension
new_shape = original_shape[:-1] + (hidden_dim,)

# Reshape and return
return output.view(new_shape)

```

```

@triton.jit
def rmsnorm(x_ptr, residual_ptr, gamma_ptr, out_ptr, num_tokens, token_stride, res_stride, hidden_dim, eps, has_residual: tl.constexpr):
    """
    Kernel which applies RMSNorm to the input tokens.

    Parameters:
    x_ptr: Pointer to the input tokens.
    residual_ptr: Pointer to the residual tokens.
    gamma_ptr: Pointer to the gamma values.
    out_ptr: Pointer to the output tokens.
    num_tokens: Number of tokens.
    token_stride: Stride between tokens.
    res_stride: Stride between residual tokens.
    hidden_dim: Hidden dimension.
    eps: Epsilon value.
    has_residual: Whether residual tokens are present.
    block_size_m: Block size for the number of tokens.
    block_size_k: Block size for the hidden dimension.
    """
    # Get 1D block id
    pid = tl.program_id(axis = 0)

    # Set epsilon value for handling zero embedding vectors
    epsilon = eps

    # Thread id with which block starts
    block_start = pid * block_size_m

    # Get the offsets for the number of tokens for a specific block
    token_offsets = block_start + tl.arange(0, block_size_m)

    # Mask for handling the final block
    token_mask = token_offsets < num_tokens

    # Get the offsets for the hidden dimension. Since the block handles entire token at once, we do not add block start.
    hidden_dim_offsets = tl.arange(0, block_size_k)

    # For handling offsets beyond hidden dimension length
    hidden_dim_mask = hidden_dim_offsets < hidden_dim

    # Load the input tokens and the gamma values
    tokens = tl.load(x_ptr + token_offsets[:, None] * token_stride + hidden_dim_offsets[None, :], mask = token_mask[:, None] * hidden_dim_mask, other = 0)
    gammas = tl.load(gamma_ptr + hidden_dim_offsets, mask = hidden_dim_mask, other = 0)

    # For handling residual
    if has_residual:
        # Load residual
        residual = tl.load(residual_ptr + token_offsets[:, None] * res_stride + hidden_dim_offsets[None, :], mask = token_mask[:, None] * hidden_dim_mask, other = 0)

        # Add residual
        tokens += residual

    # Store to be passed back for later block
    tl.store(out_ptr + token_offsets[:, None] * token_stride + hidden_dim_offsets[None, :], tokens, mask = token_mask[:, None] * hidden_dim_mask)

    # Upcast from float16 to float32 to prevent underflow
    tokens_f32 = tokens.to(tl.float32)
    gammas_f32 = gammas.to(tl.float32)

    # MS value
    var = tl.sum(tokens_f32 * tokens_f32, axis = 1) / hidden_dim + epsilon

```

```

# Inverse RMS value
inv_rms_vals = 1.0 / tl.sqrt(var)

# Normalize the input token matrix
normed_outputs_f32 = tokens * inv_rms_vals[:, None] * gammas_f32[None, :]

# Downcast to float16 to match model architecture
normed_outputs = normed_outputs_f32.to(tokens.dtype)

# Store the output
tl.store(out_ptr + token_offsets[:, None] * token_stride + hidden_dim_offsets[None, :], normed_outputs, mask = token_mask[:,

```

```

def custom_RMSNorm_cuda_forward(self, x, residual: torch.Tensor | None = None):
    """Function to replace the forward method for CUDA RMSNorm in vLLM Llama."""
    if self.variance_size_override is not None:
        return self.forward_native(x, residual)

    # Boolean to indicate prescence of residual
    has_residual = False

    # Store 3d shape for output
    original_shape = x.shape

    # Squash first two dimensions to handle token level dimensionality
    squashed_x = x.view([-1, x.shape[-1]])

    # Initialize dummy residual value
    squashed_residuais = squashed_x

    # For handling residual
    if residual is not None:
        has_residual = True
        squashed_residuais = residual.view([-1, residual.shape[-1]])

    # Get the total number of tokens and the hidden dimension
    total_tokens = squashed_x.shape[0]
    hidden_dim = squashed_x.shape[1]

    # Initialize output matrix
    output = torch.empty((total_tokens, hidden_dim), device = x.device, dtype = x.dtype)

    # For handling cases when hidden dimension is not a perfect power of 2
    block_size_k = triton.next_power_of_2(hidden_dim)

    # Initialize the block grid
    grid = lambda meta: ((triton.cdiv(total_tokens, meta['block_size_m'])),)

    # Initialize epsilon
    eps = self.variance_epsilon

    # Invoke the kernel
    rmsnorm[grid](squashed_x, squashed_residuais, self.weight.data, output, total_tokens, squashed_x.stride(0), squashed_residuais

    # Match return statement according to original function
    if residual is not None:
        return (output.view(original_shape), residual.view(original_shape))

    return output.view(original_shape)

# Invoke runtime swap
RMSNorm.forward_cuda = custom_RMSNorm_cuda_forward

```

```

# Custom function object
custom_swiglu = customSwiglu()

def patched_llama_forward(self, x):
    """Function definition based on LlamaMLP.forward."""
    # Execute the native AWQ projection
    x, _ = self.gate_up_proj(x)

    # Bypass self.act_fn(x) entirely
    x = custom_swiglu(x)

    # Finish the standard layer execution
    x, _ = self.down_proj(x)
    return x

```

```
# Execute the runtime swap
LlamaMLP.forward = patched_llama_forward
```

```
# Initialize the vLLM object
llm = LLM(model="casperhansen/llama-3-8b-instruct-awq", quantization = "awq", tensor_parallel_size = 1, enforce_eager = True)
```

[Show hidden output](#)

```
# Set parameters for the output
params = SamplingParams(temperature=0.8, max_tokens = 100)

# Get the tokenizer for applying chat template
tokenizer = llm.get_tokenizer()
```

```
# Block to fetch prompts from the alpaca dataset
from datasets import load_dataset
ds = load_dataset("tatsu-lab/alpaca", split = "train")
inputs = ds["instruction"][:200]
```

README.md: 7.47k/? [00:00<00:00, 618kB/s]

data/train-00000-of-00001-a09b74b3ef9c3b(...): 100%

24.2M/24.2M [00:01<00:00, 18.4MB/s]

Generating train split: 100%

52002/52002 [00:00<00:00, 182860.93 examples/s]

```
formatted_prompts = []
for prompt in inputs:
    # Wrap the raw text in the standard dictionary format
    message = [{"role": "user", "content": prompt}]

    # Apply the template to create the raw string Llama-3 expects
    formatted_string = tokenizer.apply_chat_template(
        conversation=message,
        tokenize=False,
        add_generation_prompt=True
    )

    formatted_prompts.append(formatted_string)
```

```
# Output for 50 prompts

# outputs = llm.generate(formatted_prompts, sampling_params = params)

# for output in outputs:
#     prompt = output.prompt
#     generated_text = output.outputs[0].text
#     print(f"Prompt: {prompt!r}, Generated text: {generated_text!r}")
```

[Show hidden output](#)

```
outputs = llm.generate(formatted_prompts, sampling_params = params)

for output in outputs:
    prompt = output.prompt
    generated_text = output.outputs[0].text
    print(f"Prompt: {prompt!r}, Generated text: {generated_text!r}")
```

Rendering prompts: 100%

200/200 [00:00<00:00, 858.38it/s]

Processed prompts: 100%

200/200 [00:44<00:00, 1.14it/s, est. speed input: 92.14 toks/s, output: 273.80 toks/s]

Prompt: '<|begin_of_text|><|start_header_id|>user<|end_header_id|>\n\nGive three tips for staying healthy.<|eot_id|><|start_header_id|>system<|end_header_id|>\n\nWhat are the three primary colors?<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nDescribe the structure of an atom.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nHow can we reduce air pollution?<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nDescribe a time when you had to make a difficult decision.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nIdentify the odd one out.<|eot_id|><|start_header_id|>system<|end_header_id|>\n\nExplain why the following fraction is equivalent to 1/4.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nWrite a short story in third person narration about a person who is afraid of heights.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nRender a 3D model of a house.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nEvaluate this sentence for spelling and grammar mistakes.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nHow did Julius Caesar die?<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nWhat is the capital of France?<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nGenerate a list of ten items a person might need for a camping trip.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nDiscuss the causes of the Great Depression.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nClassify the following into animals, plants, and minerals.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nExplain the use of word embeddings in Natural Language Processing.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nDescribe the function of a computer motherboard.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nReverse engineer this code to create a new version.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nPropose an ethical solution to the problem of data privacy.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nGenerate three verbs that mean the same as "to apologize".<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nWhat does DNA stand for?<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nCompare and contrast the Cuban Missile Crisis and the Vietnam War.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nGenerate a list of random words.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nTranscribe the following audio recording into text.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nWho is the world's most famous painter?<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nExplain the concept of cogging torque.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nLook up the boiling point of water.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nDescribe the life and reign of King Charles II.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nFind the area of a circle given its radius.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nIdentify the lines of longitude that divides North and South America.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nExplain the process of cellular respiration in plants.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nRewrite the following sentence using active voice.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nGenerate a list of adjectives that describe a person as being brave.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nOutline the consequences of deforestation.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nDevelop a plan to reduce electricity usage in a home.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nArrange the words in the given sentence to form a grammatically correct sentence.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nRewrite the given paragraph in a shorter, easier-to-understand format.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nDesign a logo for a website about recycling.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nGenerate a poem that expresses joy.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nConvert the given equation into an algebraic expression.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nBrainstorm possible solutions to reduce water pollution.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nExplain why the given definition is wrong.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nVariable x is defined as "4x + 2y = 10". Find the value of x when y = 5.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nWrite a short paragraph about the given topic.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nExplain the concept of artificial intelligence in simple terms.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nDesign an app for a delivery company.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nSummarize the given passage.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nExtract the facts from the paragraph.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nEdit the following sentence to make it more concise.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nGenerate a poem with 10 lines.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nConvert from celsius to fahrenheit.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nArrange the given numbers in ascending order.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nCalculate the total surface area of a cube with a side length of 3 units.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nWhat is the force on a 1 kg mass due to the gravitational pull of the Earth?<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nProvide one example for a cultural practice.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nGiven a set of numbers, find the maximum value.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nGive two examples of a liquid.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nWhat is the product of 6 and 2?<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nWhat type of plant is a skunk cabbage?<|eot_id|><|start_header_id|>user<|end_header_id|>

Start coding or generate with AI.

Prompt: '<|begin_of_text|><|start_header_id|>user<|end_header_id|>\n\nRewrite the given paragraph in a shorter, easier-to-understand format.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nDesign a logo for a website about recycling.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nGenerate a poem that expresses joy.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nConvert the given equation into an algebraic expression.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nBrainstorm possible solutions to reduce water pollution.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nExplain why the given definition is wrong.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nVariable x is defined as "4x + 2y = 10". Find the value of x when y = 5.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nWrite a short paragraph about the given topic.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nExplain the concept of artificial intelligence in simple terms.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nDesign an app for a delivery company.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nSummarize the given passage.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nExtract the facts from the paragraph.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nEdit the following sentence to make it more concise.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nGenerate a poem with 10 lines.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nConvert from celsius to fahrenheit.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nArrange the given numbers in ascending order.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nCalculate the total surface area of a cube with a side length of 3 units.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nWhat is the force on a 1 kg mass due to the gravitational pull of the Earth?<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nProvide one example for a cultural practice.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nGiven a set of numbers, find the maximum value.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nGive two examples of a liquid.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nWhat is the product of 6 and 2?<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nWhat type of plant is a skunk cabbage?<|eot_id|><|start_header_id|>user<|end_header_id|>